Warehouse: The End-to-End Pipeline Process

Aliakbar Ghafari

Ironhack, AI & DataScience Course

Project of Week 13

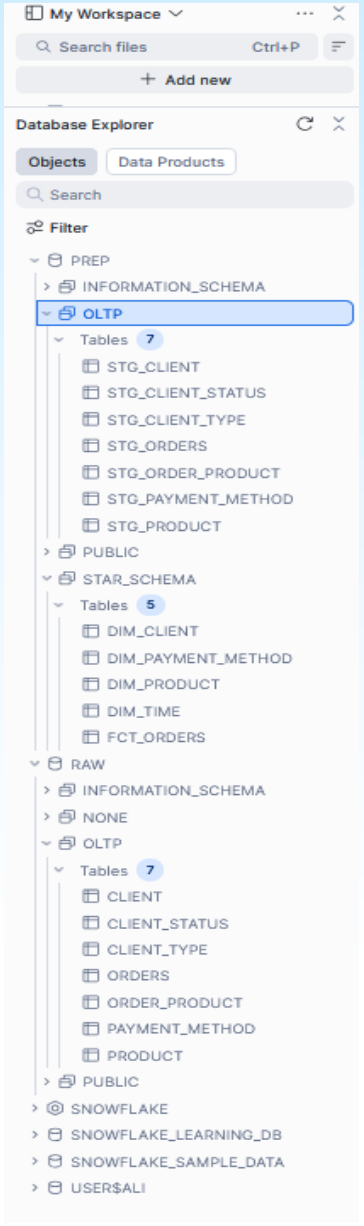
13.2.2026

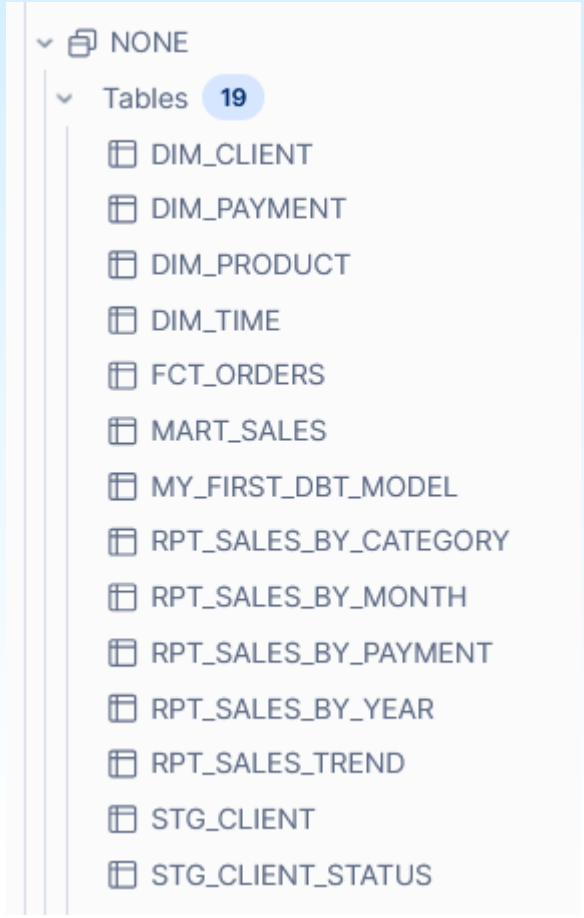
Introduction

End-to-End Data Warehouse with Snowflake & dbt

Project Structure

```
models/
│
├── sources.yml                ← RAW (OLTP)
│
├── 01-staging/                ← Clean, rename, standardize
│   ├── stg_client.sql
│   ├── stg_client.yml
│   ├── stg_client_status.sql
│   ├── stg_client_status.yml
│   ├── stg_client_type.sql
│   ├── stg_client_type.yml
│   ├── stg_orders.sql
│   ├── stg_orders.yml
│   ├── stg_order_product.sql
│   ├── stg_order_product.yml
│   ├── stg_product.sql
│   ├── stg_product.yml
│   ├── stg_payment_method.sql
│   └── stg_payment_method.yml
│
├── 02-prep/                   ← Dimensions + Fact (Star Schema)
│   ├── dim_client.sql
│   ├── dim_client.yml
│   ├── dim_product.sql
│   ├── dim_product.yml
│   ├── dim_payment.sql
│   ├── dim_payment.yml
│   ├── dim_time.sql
│   ├── dim_time.yml
│   ├── fct_orders.sql
│   └── fct_orders.yml
│
└── 03-marts
    ├── mart_sales.sql
    ├── mart_sales.yml
    ├── rpt_sales_by_category.sql
    ├── rpt_sales_by_month.sql
    ├── rpt_sales_by_payment.sql
    ├── rpt_sales_by_year.sql
    ├── rpt_sales_trend.sql
```





End-to-End Data Workflow

Step 1: Source Data Analysis
- Identified operational tables (clients, orders, products, payments).
- Reviewed data types, keys, and date formats in Snowflake.

Step 2: Staging Layer (dbt)
- Created **staging models**
- Renamed columns, fixed date formats, and removed inconsistencies.

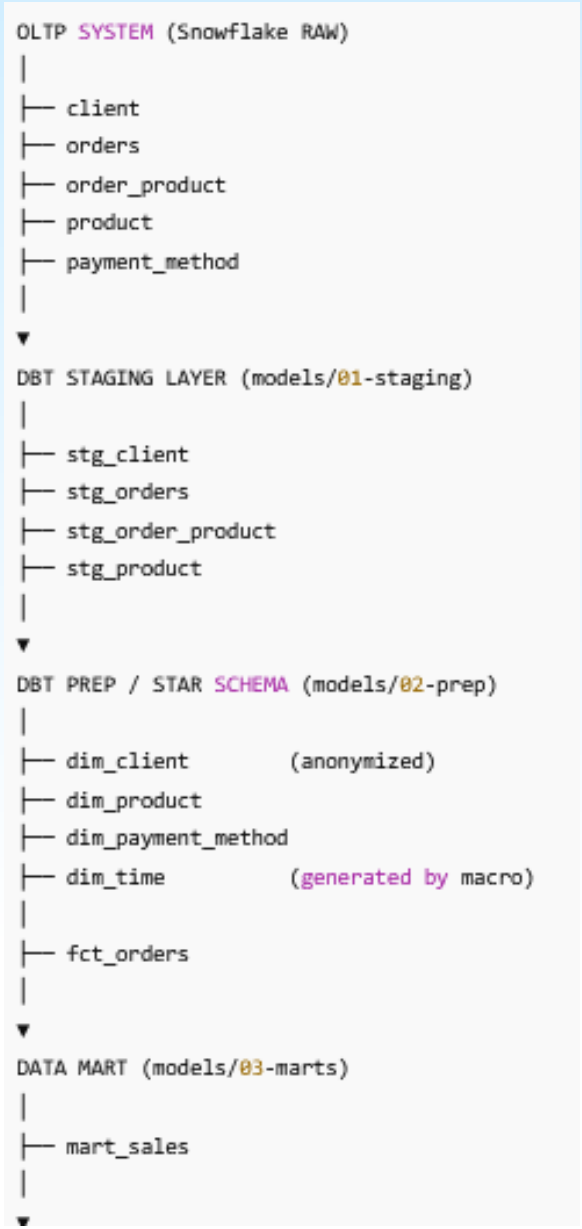
Step 3: Dimensional Modeling
- Built **dimension tables** (Time, Client, Product, Payment).
- Created **fact table (FCT_ORDERS)** with sales metrics.
- Applied **star schema** design for analytics.

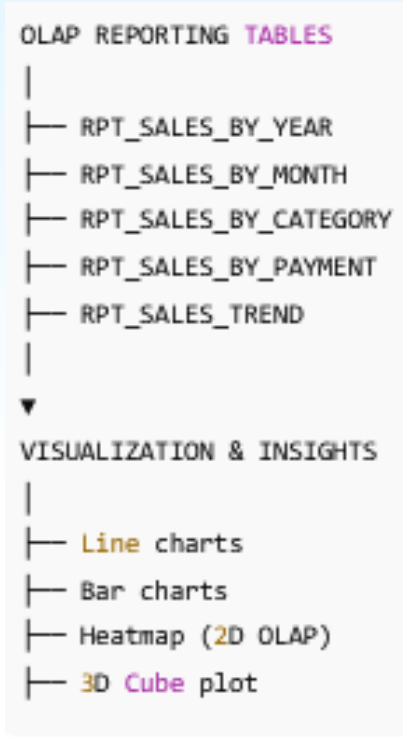
Step 4: Data Quality & Governance
- Added **dbt tests** (not-null, relationships).
- Anonymized sensitive client data using **dbt macros (hashing)**.

Step 5: Data Mart Creation
- Built **Sales Mart** optimized for reporting.
- Aggregated sales by time, product, and payment method.

Step 6: Reporting Tables
- Created reporting tables:

Step 7: Visualization & Insights
- Exported results to Python.
- Created **2D heatmap** and **3D sales plots**.
- Analyzed trends, seasonality, and business performance.





Pipeline and Architecture

Why This Pipeline Was Implemented:

- The pipeline converts OLTP e-commerce data into an OLAP-optimized structure, enabling fast reporting, historical analysis, and consistent business metrics.
- This pipeline demonstrates how raw data can be transformed into governed, analytical assets that deliver actionable business insights

Why dbt and Layered Architecture:

- dbt was used to apply modular, testable transformations and enforce a clear separation between staging, modeling, and reporting layers
- **Architecture: OLTP → Staging → Star Schema → Sales Mart → OLAP Reporting**

Why Star Schema Was Used:

The star schema was chosen to simplify analytical queries, improve performance, and align with Kimball's dimensional modeling methodology

Star Schema:

- Star schema enables business users to easily analyze sales across time, product, and payment dimensions without complex joins
- Fact tables store measurable events, while dimension tables provide descriptive context for multidimensional analysis

Why OLAP Reporting Tables Were Created:

- Pre-aggregated OLAP reporting tables were created to enable fast, consistent, and reusable analytics.

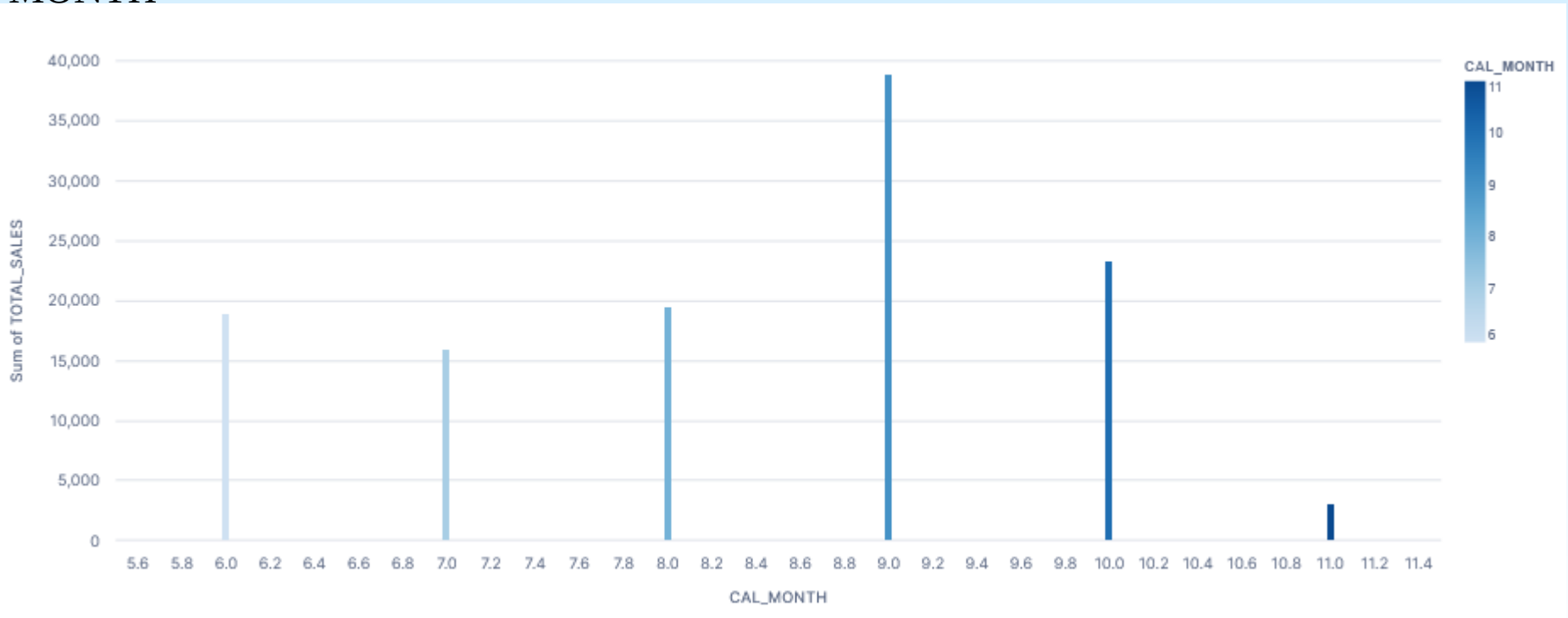
Total sales by month and year (OLAP)

Table name : RPT_SALES_BY_ MONTH

Sql:

```
SELECT    cal_year,
cal_month,
total_salesFROM
RPT_SALES_BY_MO
NTHORDER BY
cal_year,
cal_month;
```

This query presents total sales by month and year to highlight seasonal and temporal sales trends.



By analyzing total sales per month and year, the chart helps identify peak seasons and periods of growth or decline.

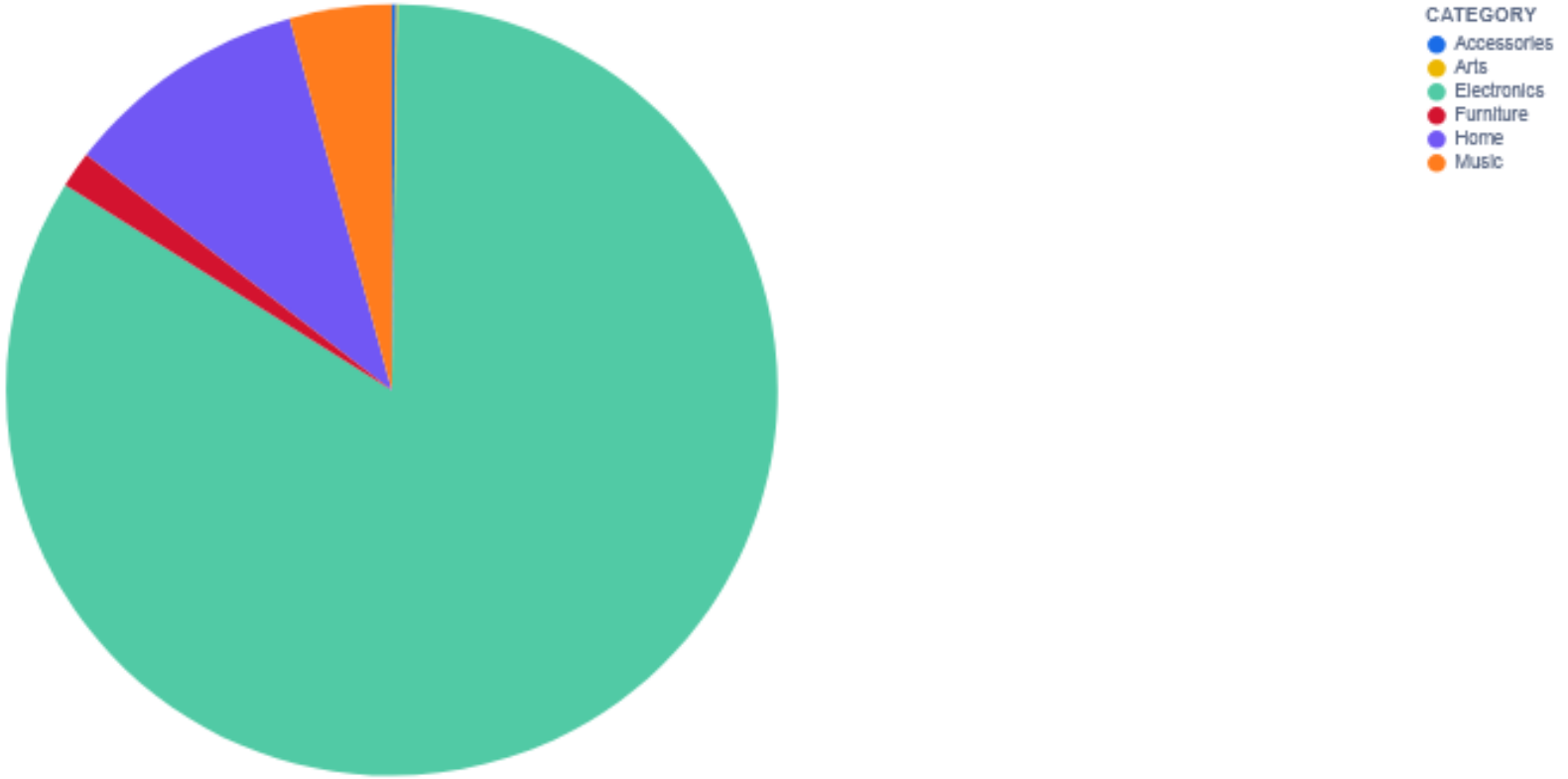
Total sales across product categories (OLAP)

Table name: RPT_SALES_BY_CATEGORY

Sql:

```
SELECT    category,
total_salesFROM
RPT_SALES_BY_CATE
GORYORDER BY
total_sales DESC;
```

This query compares total sales across product categories, highlighting which categories contribute the most to overall revenue



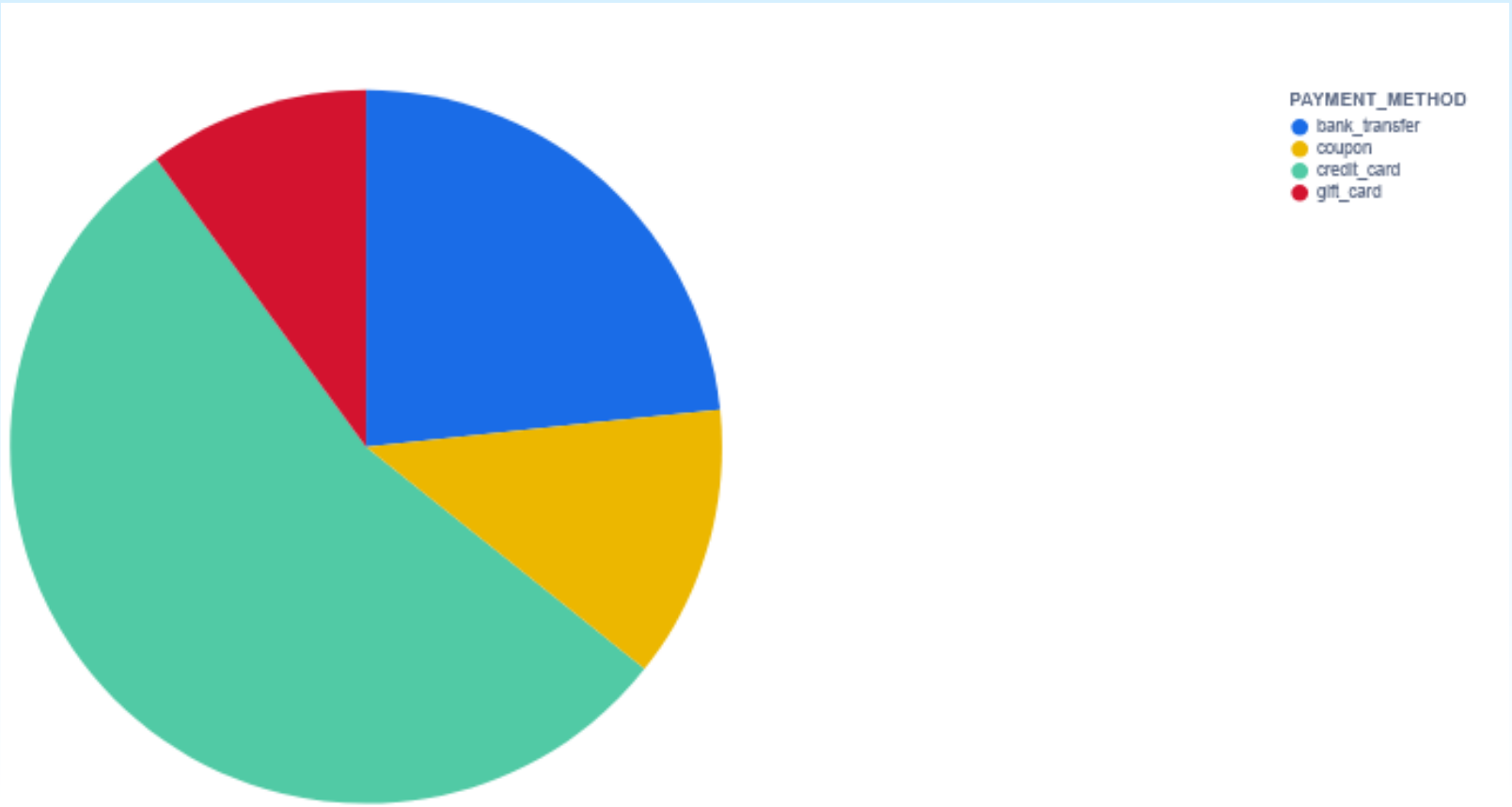
By ranking product categories by total sales, the analysis identifies high-performing categories and opportunities for portfolio optimization.

Total sales by payment method (OLAP)

Table name
RPT_SALES_BY_PAYMENT

Sql:
SELECT payment_method,
total_salesFROM
RPT_SALES_BY_PAYMENTORDER BY
total_sales DESC;

This analysis shows total sales by payment method, revealing which payment options customers prefer.

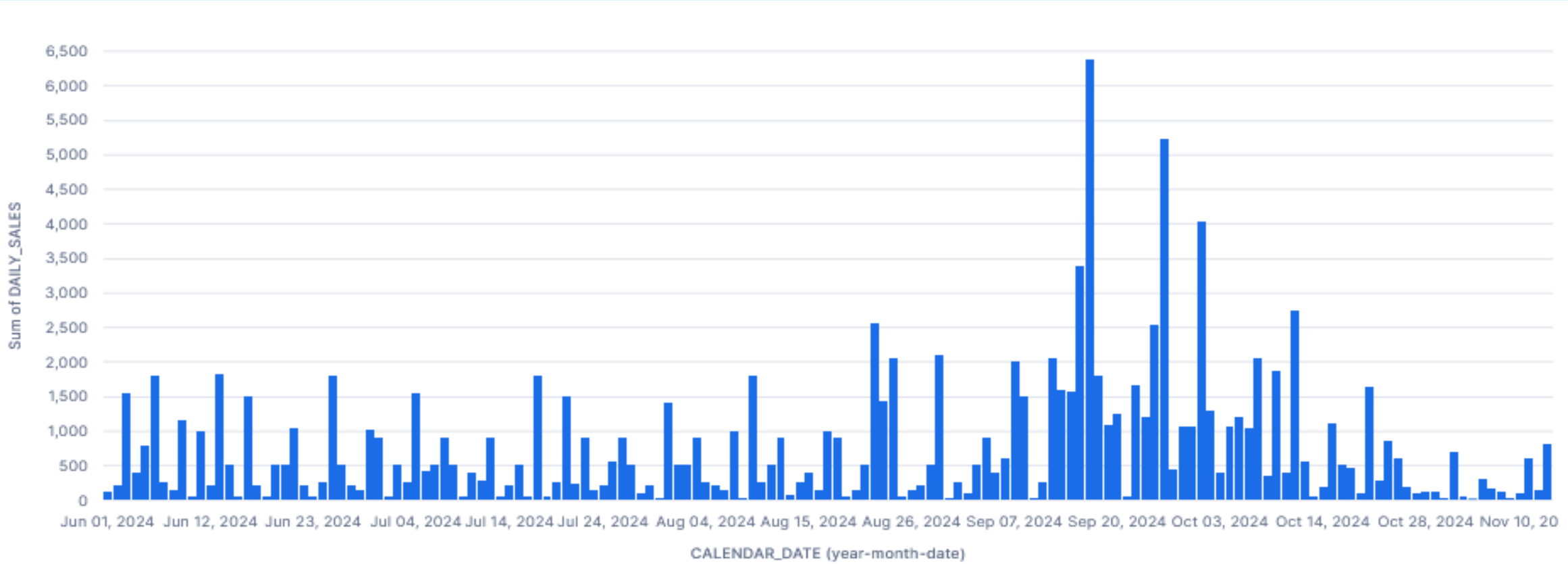


This chart compares total sales across different payment methods

Sales Trend Over Time (OLAP)

Table name: RPT_SALES_TREND

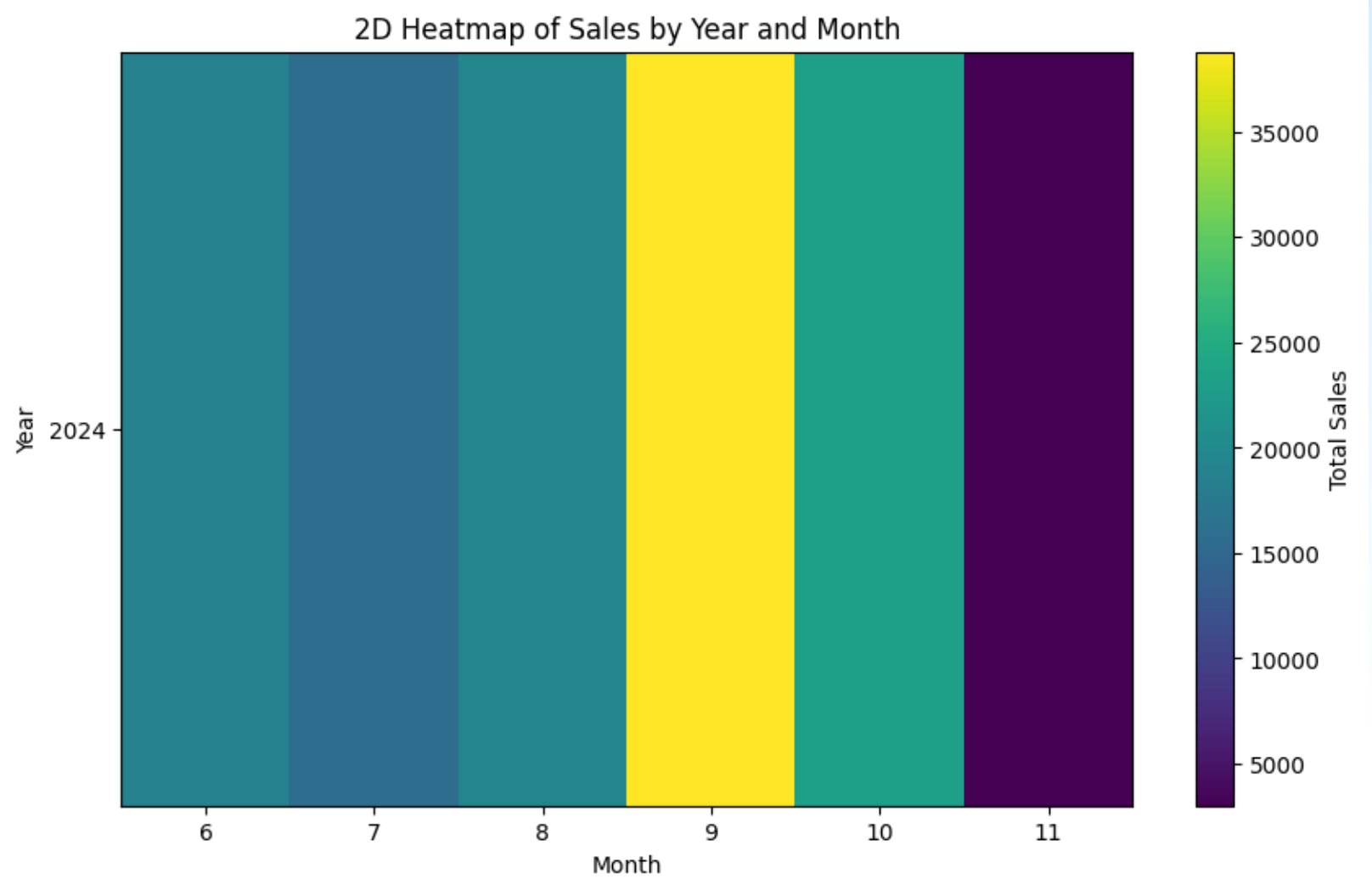
SELECT calendar_date, daily_salesFROM RPT_SALES_TRENDORDER BY calendar_date;



This chart shows the daily sales trend over time, helping to identify growth patterns, seasonality, and demand fluctuations.

Monthly Sales Summary: 2D HEAT MAP (OLAP)

Sql:
SELECT cal_year, cal_month, total_salesFROM RPT_SALES_BY_MONTHORDER BY cal_year, cal_month;



This visualization summarizes total sales by month and year, enabling comparison of monthly performance across different periods.

Summary

- This project transforms OLTP e-commerce data into an OLAP-ready sales data warehouse with governance, reporting, and analytical insights.

- **Architecture: OLTP → Staging → Star Schema → Sales Mart → OLAP Reporting**
- OLAP Reporting Tables:

  Pre-aggregated tables for fast analytics:

  - RPT_SALES_BY_YEAR
  - RPT_SALES_BY_MONTH
  - RPT_SALES_BY_CATEGORY
  - RPT_SALES_BY_PAYMENT
  - RPT_SALES_TREND

Acknowledgements:

Moushumi and Rami for support

Thanks for your attention